Ôn kiểm tra khóa 2

Một pipeline chạy, đo và nộp được

Đề ôn

Ảnh hoạt động, ba nhóm, metric macro-F1.

Một hàng dự đoán cho mỗi ID.

Ảnh cùng buổi giống nhau: test là buổi mới thì validation cũng là buổi mới.

Buổi này đi lại cả quy trình bằng một ví dụ chạy được.

Mục tiêu buổi ôn

- Dựng baseline theo đúng split và metric.
- Debug một batch bằng phép kiểm nhỏ.
- Chọn phương pháp bằng validation có ý nghĩa.
- Lưu, tải lại và kiểm file output.

01

Từ đề đến baseline

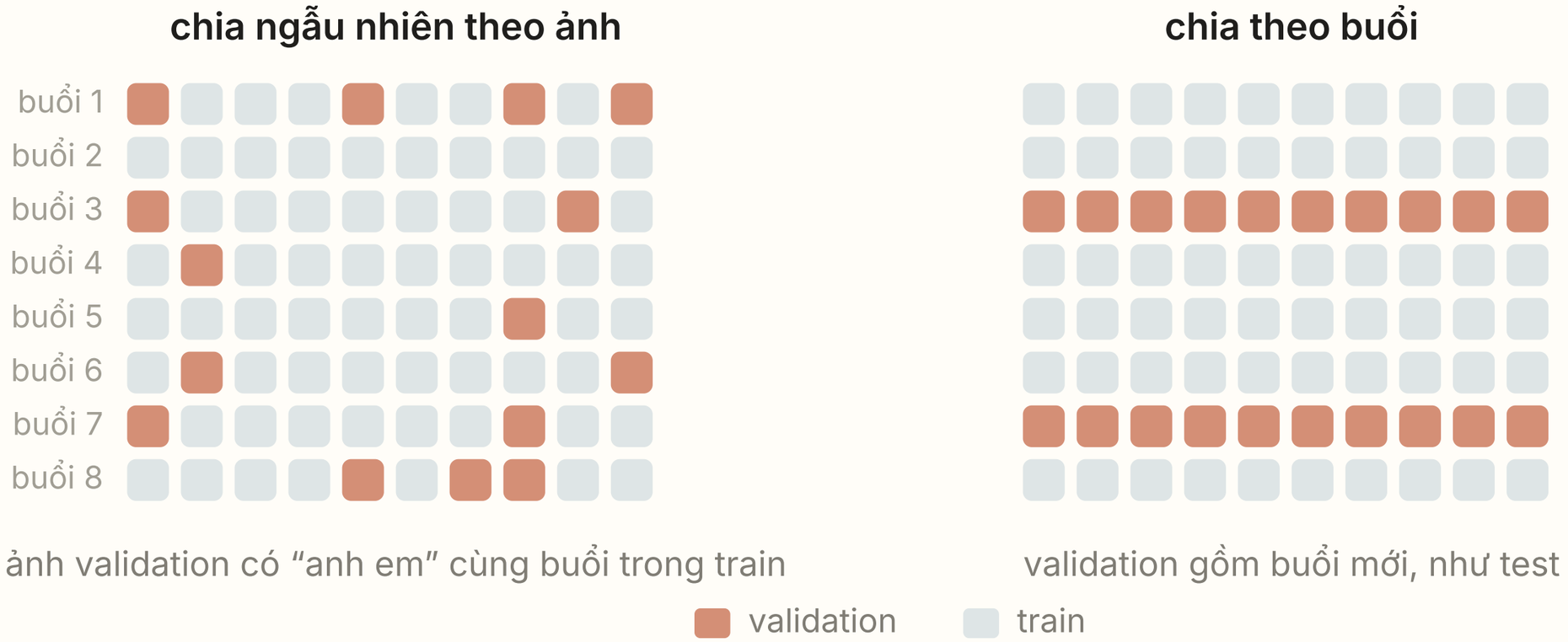
Chạy vài hàng từ đầu đến cuối

- Theo dõi đủ các bước: input, tensor, logits, nhãn và CSV.
- Giữ ID qua mọi bước.
- Chia theo đơn vị cần tổng quát hóa.

Baseline nộp được trước, model lớn sau.

`weights=None` là mạng chưa học, không phải pretrained.

Chia theo đơn vị của test



Hai cách chia cùng 8 buổi ảnh: chia ngẫu nhiên rải ảnh validation vào mọi buổi; chia theo buổi giữ riêng cả buổi, đúng tình huống test là buổi mới.

HỎI NHANH

Ảnh đến từ 40 buổi sinh hoạt, mỗi buổi 30 ảnh, và test là 10 buổi tổ chức sau đó. Chia ngẫu nhiên theo ảnh thì macro-F1 trên validation là 0.93, chia theo buổi thì là 0.71. Con số nào gần với điểm test hơn?

A 0.71, vì validation cũng gồm những buổi chưa thấy

B 0.93, vì validation ngẫu nhiên có đủ mọi buổi

C 0.93, vì hai cách chia dùng chung một kiến trúc model

D 0.82, vì trung bình hai cách chia cho ước lượng ổn hơn

HỎI NHANH

Máy thi không có Internet. Code gọi `resnet50(weights=None)`, freeze toàn bộ rồi train head trên 600 ảnh, và macro-F1 validation chỉ được 0.35. Nên nghi gì trước tiên?

A Head quá nhỏ, cần thêm hai lớp ẩn

B Backbone chưa học gì, chỉ là trọng số ngẫu nhiên

C Learning rate của head quá lớn với 600 ảnh

D Macro-F1 luôn thấp hơn accuracy, nên số này bình thường

02

Debug và chọn phương pháp

Một batch trước khi chạy dài

`logits`: `(B,3)`; `labels`: `(B,)`, kiểu long.

Cross-entropy nhận logits.

Loss hữu hạn; gradient ở đúng phần cần học.

Head-only: so `state_dict` encoder trước/sau.

BatchNorm: buffer đổi ở train mode dù đã freeze.

LoRA: gradient của A bằng 0 ở bước đầu là đúng.

HỎI NHANH

Model ba lớp kết thúc bằng `nn.Softmax(dim=1)`, rồi đưa output vào `CrossEntropyLoss`. Thử cho học thuộc 8 mẫu: accuracy lên 1.0, nhưng loss dừng ở 0.551 và không giảm thêm. Hiểu thế nào?

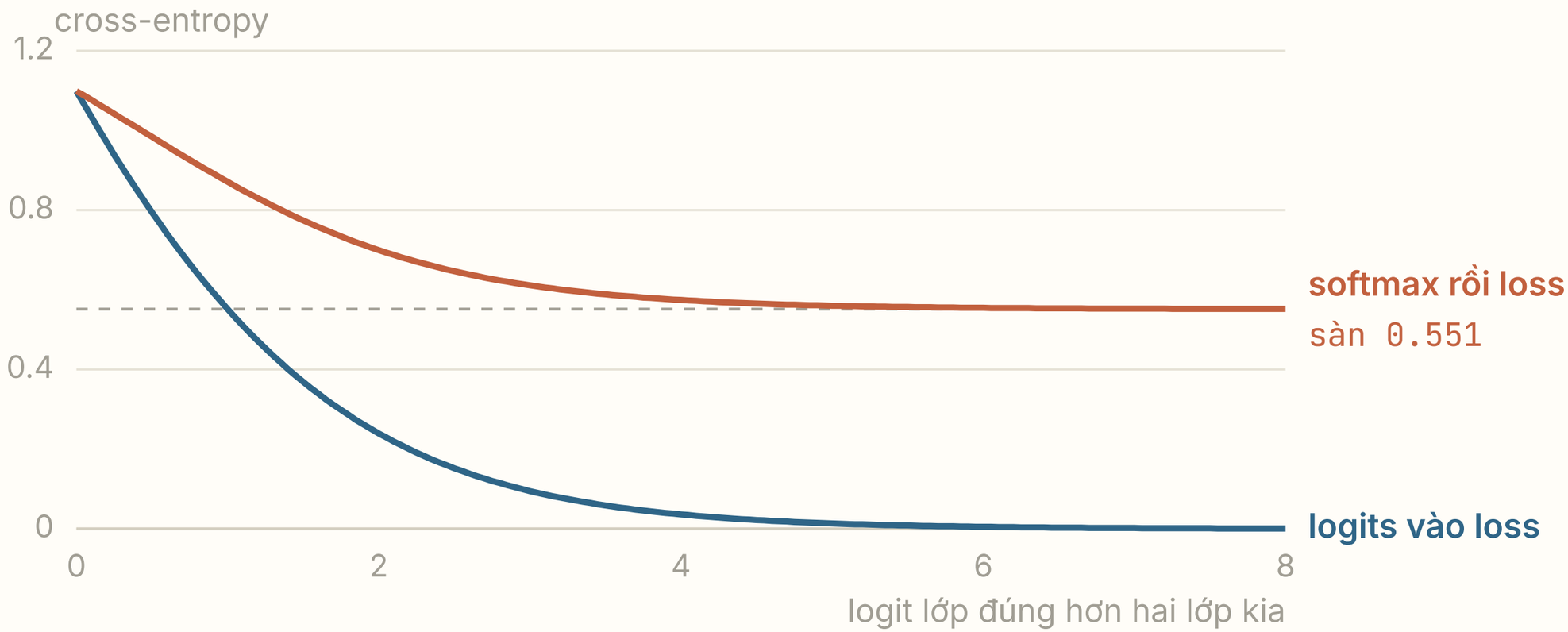
A Learning rate quá nhỏ, loss chưa kịp về 0

B 8 mẫu quá ít, loss không thể thấp hơn

C Nhãn bị lệch nên loss không xuống thêm

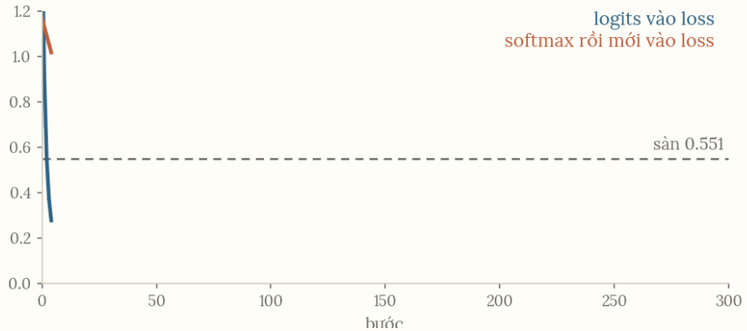
D Softmax bị áp hai lần, nên loss có một mức sàn 0.551

Sàn của softmax hai lần

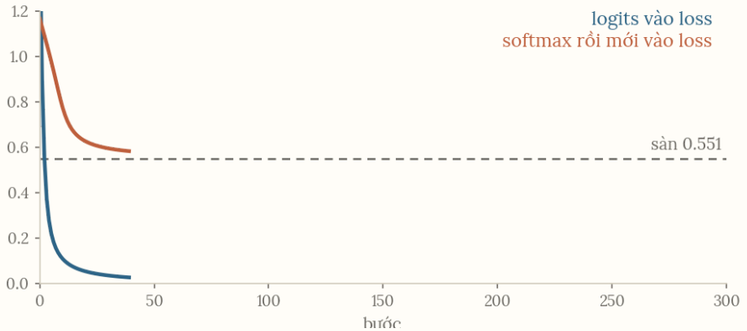


Cross-entropy của một mẫu ba lớp khi logit lớp đúng tăng dần: đưa logits vào loss thì về 0; áp softmax trước thì loss kẹt ở sàn $-\ln(e/(e+2)) \approx 0.551$.

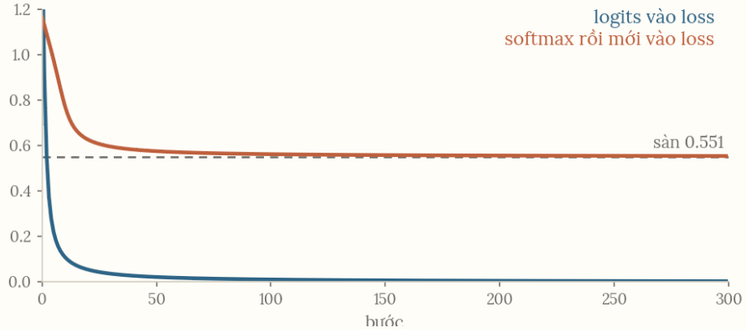
Hai cách tính loss khi train



bước 5: loss đúng 0.279, loss khi softmax hai lần 1.019



bước 40: loss đúng 0.028, loss khi softmax hai lần 0.585



bước 300: loss đúng 0.004, loss khi softmax hai lần 0.555

Head của buổi kiểm tra học ba mẫu đầu (mỗi lớp một mẫu) với hai cách tính loss: đưa logits vào thì loss về gần 0; softmax trước thì loss chỉ tiến sát sàn 0.551.

HỎI NHANH

LoRA một chiều: $y = (w_0 + ba)x$, với $w_0 = 1$ đã freeze, $a = 0.5$, $b = 0$. Mẫu có $x = 1$, target $t = 3$, loss $(y - t)^2$, SGD với learning rate 0.1. Ở bước 1, gradient của a bằng 0. Ở bước 2, gradient của a bằng bao nhiêu?

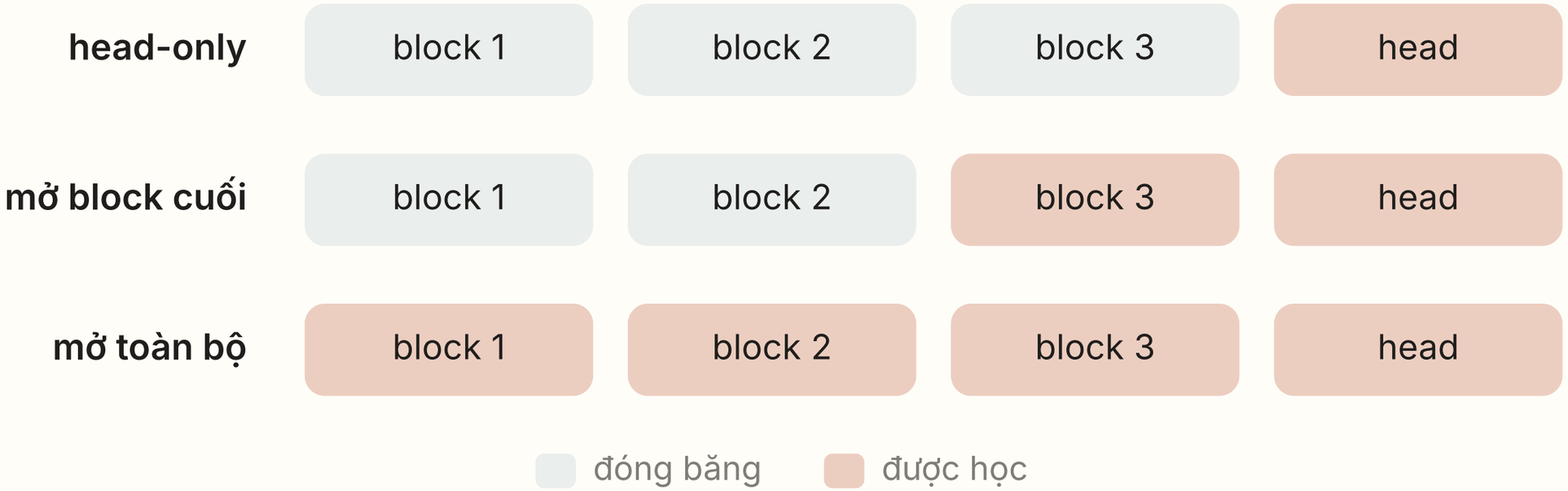
A 0

B -2

C -0.76

D -1.9

Ba mức mở tham số



Ba mức mở tham số, thử theo thứ tự: chỉ head, thêm block cuối, rồi toàn bộ encoder.

Bốn câu hỏi khác nhau

- Head-only: embedding đọc được nhãn?
- Fine-tuning: mở encoder có ích?
- LoRA: phần sửa nhỏ có đủ?
- Zero-shot: kiến thức nguồn tạo mốc gì?

Mỗi thay đổi có một lý do từ lỗi hoặc giới hạn hiện tại.

Đo lại trên cùng split, so với mốc cũ.

HỎI NHANH

Head-only cho macro-F1 0.74 trên train và 0.72 trên validation, và train thêm epoch cũng không đổi gì. Bước tiếp theo hợp lý nhất là gì?

A Train head thêm 50 epoch tới khi train đạt 1.0

B Mở block cuối hoặc thêm LoRA, rồi so trên cùng cách chia

C Đổi sang accuracy, vì macro-F1 khó tăng hơn

D Gộp validation vào train để head có thêm dữ liệu

Ghi một thí nghiệm dùng được

Split, processor, model/revision, seed.

Batch, precision, learning rate, phần mở.

Validation metric, checkpoint chọn.

Thời gian và bộ nhớ thực đo.

Giữ bản có thể nộp trước khi thử thay đổi lớn.

HỎI NHANH

Lần chạy mới vừa mở thêm hai block, vừa tăng learning rate từ 10^{-4} lên $3 \cdot 10^{-4}$, và macro-F1 validation tăng từ 0.72 lên 0.75. Kết luận nào có căn cứ?

A Mở thêm block là nguyên nhân của cải thiện

B Learning rate lớn hơn là nguyên nhân của cải thiện

C Hai thay đổi đóng góp như nhau, mỗi cái 0.015

D Cấu hình mới tốt hơn, nhưng chưa tách được nguyên nhân

03

Đọc kết quả và chốt output

Train giảm, validation tăng

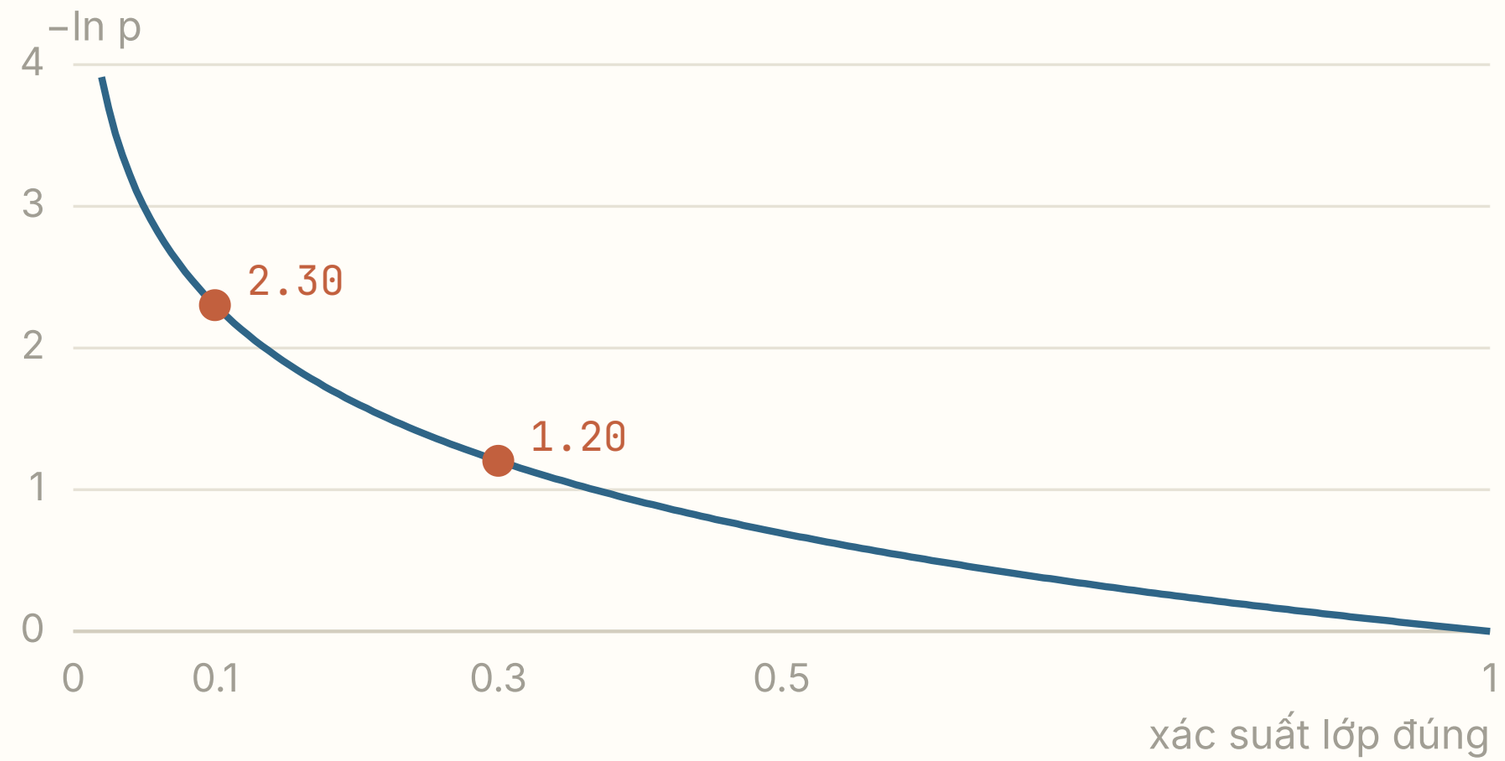
Validation loss đã qua đáy, train loss vẫn giảm.

Giữ checkpoint theo tiêu chí đã chọn; kiểm augmentation, mức mở, regularization.

Accuracy chỉ nhìn argmax, loss nhìn xác suất lớp đúng.

Mẫu sai: xác suất đúng 0.3 → 0.1, loss 1.20 → 2.30, accuracy không đổi.

Loss nhìn xác suất, accuracy nhìn argmax



cả hai lúc đều đoán sai

accuracy không đổi

loss mẫu này

1.20 → 2.30

Một mẫu sai có xác suất lớp đúng giảm từ 0.3 xuống 0.1: argmax vẫn sai nên accuracy giữ nguyên, còn loss của mẫu tăng từ 1.20 lên 2.30.

HỎI NHANH

Từ epoch 3 tới epoch 5, validation accuracy giữ ở 0.84, validation loss tăng từ 0.41 lên 0.58, còn train loss giảm từ 0.20 xuống 0.05. Hiểu thế nào?

A Model ngày càng tự tin vào những mẫu sai, tức đang overfit

B Loss bị tính sai, vì accuracy không đổi thì loss phải đứng yên

C Model chưa học xong, cần train thêm tới khi accuracy tăng

D Validation quá khó, train loss mới là thước đo đúng

Tải lại trong process mới

- Model, processor và bảng nhãn đã lưu.
- Đúng chế độ eval, tắt gradient khi suy luận.
- So vài output trước/sau lưu.
- Dự đoán đủ dữ liệu; kiểm ID, số hàng và nhãn.

HỎI NHANH

Trong process mới, cho vài ảnh mẫu đi qua processor và checkpoint đã lưu: logits khớp đến từng số với trước khi lưu. Vậy mà file nộp sai nhãn hàng loạt. Code dựng lại `class_names` từ `os.listdir` của thư mục ảnh. Lỗi nằm ở đâu?

A Trọng số nạp sai vì thiếu `strict=True`

B Processor chuẩn hóa ảnh khác lúc train

C Bảng tên lớp dựng từ `os.listdir` bị lệch thứ tự

D Model còn ở chế độ train, nên dropout làm đổi dự đoán

Hai phép kiểm trước khi chốt

Trước khi nộp, kiểm metric trên một ví dụ nhỏ và tải lại checkpoint.

Hai phép kiểm này bắt lỗi mục tiêu đánh giá và lỗi phụ thuộc vào session.

Đọc đúng metric

Head nhỏ có trọng số đặt tay.

Target: `[0, 1, 2, 1]`.

Dự đoán: `[0, 1, 2, 0]`.

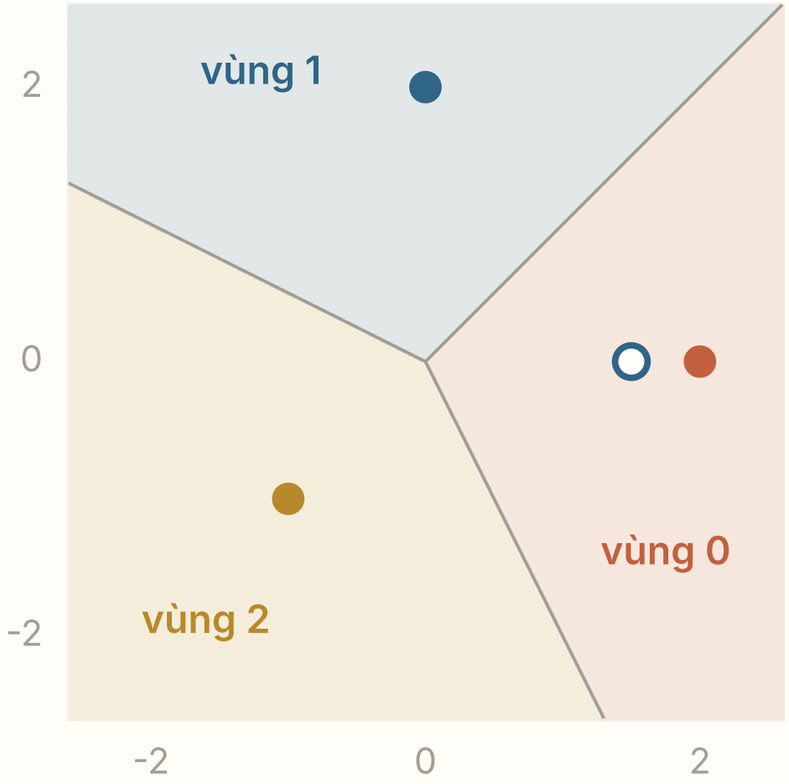
Accuracy $= 3/4$.

F1 từng lớp: $2/3$, $2/3$, 1.

Macro-F1 $= 7/9$.

Cùng accuracy, macro-F1 vẫn có thể khác.

Vì sao mẫu cuối sai




```
x = (1.5, 0)
logits 1.5, 0, -1.5
dự đoán 0, nhãn thật 1
```

biên: nơi hai logit bằng nhau
ba tia đi qua gốc

Ba vùng quyết định của head đặt tay. Mẫu (1.5, 0) có nhãn 1 (vòng rỗng) nằm trong vùng của lớp 0.

HỎI NHANH

Cùng target `[0, 1, 2, 1]`. Head A dự đoán `[0, 1, 2, 0]`, head B dự đoán `[0, 1, 1, 1]`. Cả hai có accuracy 0.75. Macro-F1 của head nào cao hơn?

A A, với 0.778, còn B chỉ 0.600

B B, với 0.800, còn A chỉ 0.778

C Bằng nhau, vì accuracy bằng nhau

D B, vì B đoán đúng cả hai mẫu lớp 1

Dựng đối tượng mới để tải lại

```
restored.load_state_dict(saved["state_dict"])
restored.eval()
with torch.inference_mode():
    restored_logits = restored(features)
assert torch.equal(logits, restored_logits)
```

`BytesIO` đóng vai tệp trong bộ nhớ.

Trong bài thi: ghi tệp thật, tải lại ở tiến trình mới.

ID giữ thứ tự input: 43, 41, 42, 44.

HỎI NHANH

Máy chấm bỏ qua cột `id`, chấm theo thứ tự hàng của file nộp mẫu `41, 42, 43, 44`, với nhãn thật lần lượt là 1, 2, 0, 1. Nộp nguyên file của ví dụ, có thứ tự `43, 41, 42, 44`, thì accuracy bằng bao nhiêu?

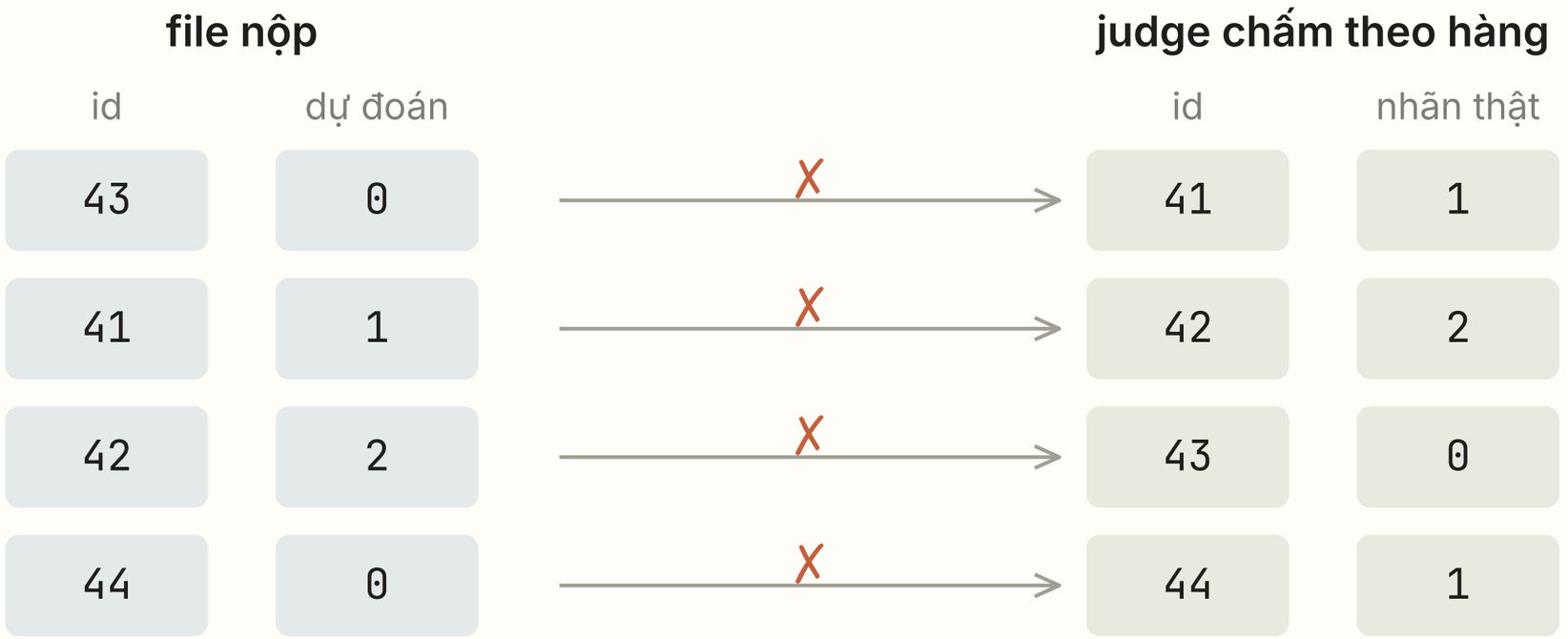
A 0.75, vì dự đoán không thay đổi

B 0.50, vì chỉ hai hàng đầu bị đảo

C 0.25, vì hàng ID 44 đúng vị trí

D 0, vì cả bốn hàng đều sai

Chấm theo hàng hay theo id



cùng nội dung: chấm theo id được 0.75, chấm theo hàng được 0

File nộp giữ thứ tự input 43, 41, 42, 44; judge bỏ cột id và chấm theo thứ tự 41, 42, 43, 44 nên cả bốn hàng đều sai.

Khi đi thi

- Đọc metric và đơn vị chia trước khi viết model.
- Nộp một bản baseline sớm, so mọi thử nghiệm với nó.
- Nạp trọng số từ tệp được cấp; `weights=None` là mạng ngẫu nhiên.
- Debug một batch: shape, loss, gradient, optimizer, BatchNorm.
- Tải lại ở process mới rồi mới nộp.

04

Luyện đề

LUYỆN ĐỀ 1

Mạng 10 lớp, mỗi lớp một neuron sigmoid: $o_i = \sigma(w_i o_{i-1} + b_i)$, với $o_0 = x$ và mọi $w_i = 1$. Cận trên của $|\partial o_{10}/\partial w_1|$ và của $|\partial o_{10}/\partial w_{10}|$ là gì?

A Với w_1 là $|x|/4$, với w_{10} là $1/4^{10}$

B Cả hai đều là $|x|/4^{10}$

C Với w_1 là $|x|/40$, với w_{10} là $1/4$

D Với w_1 là $|x|/4^{10}$, với w_{10} là $1/4$

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q4.5

LUYỆN ĐỀ 2

Linear attention cần một ánh xạ hữu hạn chiều ϕ sao cho $\phi(q)^\top \phi(k) \approx \exp(q^\top k)$, với hệ số $1/\sqrt{d}$ đã gộp vào q, k. Dùng Taylor bậc hai $\exp(x) \approx 1 + x + x^2/2$ và $(q^\top k)^2 = \sum_{a,b} q_a q_b k_a k_b$, ta lấy $\phi(q)$ gồm số 1, d thành phần q_a, và d^2 tích $q_a q_b/\sqrt{2}$. Với $d = 64$, ϕ có bao nhiêu chiều ℓ, và khi nào chi phí $O(T\ell d)$ thấp hơn $O(T^2 d)$ của attention chuẩn?

A $\ell = 129$, khi $T > 129$

B $\ell = 4161$, với mọi T, vì chi phí tuyến tính theo T

C $\ell = 4161$, khi $T > 4161$

D $\ell = 65$, khi $T > 65$

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 1 · Part 1.1.6

LUYỆN ĐỀ 3

Validation có 100 ảnh: 80 ảnh nhóm A, 12 ảnh nhóm B, 8 ảnh nhóm C. Một baseline luôn đoán A. Accuracy và macro-F1 của nó bằng bao nhiêu?

A Accuracy 0.80, macro-F1 0.80

B Accuracy 0.80, macro-F1 0.89

C Accuracy 0.80, macro-F1 0.30

D Accuracy 0.33, macro-F1 0.30

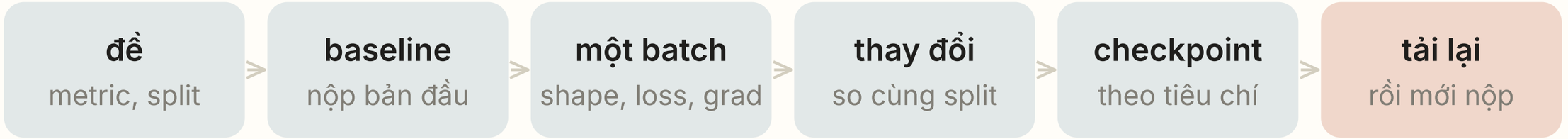
PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q5.1

Chốt quy trình

Đọc đề → split có lý do → baseline nộp được → debug một batch → chọn cách dùng pretrained → chọn checkpoint → tải lại → kiểm file.

Mỗi bước để lại một bằng chứng: điểm validation, shape, output, schema.

Thứ tự làm bài



Quy trình ôn khóa 2: đọc đề, nộp baseline, debug một batch, thử từng thay đổi trên cùng split, chọn checkpoint, tải lại rồi nộp.

Sau buổi ôn

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy lại ba ví dụ: mạng nhỏ, feature extraction và LoRA; với mỗi ví dụ, giải thích một lỗi không báo exception.